

TRA Quality Scoring — How the Tool Measures Quality

1. What the tool does

- Reads a **SoCoSi TRA JSON** export (Software-component *or* System-deployment schema).
- Computes an interview-derived **Core Metric set (CM-01 ... CM-36)**.
- Groups metrics into **5 quality dimensions**.
- Produces a **status-weighted overall score** + an interactive HTML dashboard.

No third-party dependencies — pure Python standard library; charts via Chart.js (CDN).

2. The 5 quality dimensions

Dimension	What it asks
Formal Completeness	Are all required fields, sections and assumptions filled in?
Coverage	Are all components, interfaces, assets and protection goals analysed?
Consistency & Traceability	Are ratings coherent, justified and free of duplicates?
Comprehensibility	Are threats described clearly, not vaguely or in keywords?
Process & Governance	Workshops, living-document maintenance, approval, evidence.

Each metric belongs to exactly one dimension.

3. Every metric gets a traffic-light status

Status	Badge	Colour	Meaning
ok	GOOD	 green	Passes the quality bar
warn	REVIEW	 amber	Borderline — needs a human look
bad	CRITICAL	 red	Fails the quality bar
na	N/A	 grey	Nothing to measure in <i>this</i> TRA

The status — not the raw percentage — drives the final score.

4. How **ok / warn / bad** is decided

Percentage metrics (higher is better, e.g. coverage):

- $\geq 90\%$ →  ok
- $\geq 60\%$ →  warn
- else →  bad

"Lower is better" metrics (e.g. vagueness):

- $\leq 10\%$ →  ok · $\leq 40\%$ →  warn · else →  bad

Count metrics (e.g. placeholder hits, duplicate pairs):

- 0 →  ok · 1-5 →  warn · > 5 →  bad

5. The N/A status — why it matters

na is not a zero. It means the metric *cannot be assessed* because the required data is absent from this particular TRA.

Examples

- CM-08 *external interface coverage* → N/A when no network-facing interfaces exist.
- CM-09 *communication coverage* → N/A when there are no inter-component comms.
- CM-19 *rating consistency* → N/A when there are no near-duplicate threats.
- CM-01 *workshop participation* → N/A when the schema has no workshop data.

Key rule: N/A metrics are listed in the report but **excluded from every average** — they never raise or lower a score. This avoids unfairly penalising a TRA for something that genuinely does not apply.

6. The 100 / 60 / 20 status weights

Instead of averaging raw percentages, each status is converted to a **fixed weight**:

```
STATUS_WEIGHT = {"ok": 100.0, "warn": 60.0, "bad": 20.0}
```

Status	Weight	Why
ok (GOOD)	100	Full credit
warn (REVIEW)	60	Partial credit — passing but flagged
bad (CRITICAL)	20	Almost no credit, but not literally zero
na	<i>excluded</i>	Not counted at all

Why weights, not raw %? It rewards *crossing quality thresholds*, matching how a reviewer thinks: "good, borderline, or broken?" — not chasing exact decimals.

7. The averaging formula

For any group of metrics (one dimension, or all of them):

$$\text{score} = \frac{\sum_{m \in \text{scored}} \text{weight}(m)}{|\text{scored}|}$$

where *scored* = metrics with status ok/warn/bad (N/A dropped).

Worked example — a dimension with 5 metrics: 3 ok, 1 warn, 1 na

$$\frac{100 + 100 + 100 + 60}{4} = \frac{360}{4} = 90\%$$

The N/A metric is dropped → divide by **4**, not 5.

8. Two levels of scoring — the idea

Think of it like **grading a student**:

Level	Analogy	What it averages
Level 1 — Metric	a single exam question	one check → ok / warn / bad
Level 2a — Dimension	one subject's grade	all metrics <i>in that dimension</i>
Level 2b — Overall	the final GPA	<i>all</i> metrics together

Level 1 turns each check into a status. Level 2 **rolls those statuses up** into dimension scores and one overall score — using the *same* 100/60/20 averaging.

8a. Two-level scoring — worked example

Level 1 — grade each metric (status → weight):

Metric	Status	Weight
CM-03 Field completion	 ok	100
CM-04 Placeholders	 ok	100
CM-05 Assumptions validated	 warn	60

Level 2a — the *Formal Completeness* dimension score:

$$\frac{100 + 100 + 60}{3} = \frac{260}{3} = 86.7\%$$

Level 2b — repeat for all 5 dimensions, then average **every metric** the same way to get the single **Overall TRA Quality** % shown in the hero box.

8b. Why two levels (and different thresholds)

- **Dimension scores** tell you *where* the TRA is weak (e.g. Coverage  but Process & Governance ) — useful for **diagnosis**.
- **Overall score** gives one **headline number** for management.

Each rolled-up score gets its **own** traffic light — but with *more lenient* thresholds than a single metric:

	 ok	 warn	 bad
Single metric	≥ 90	≥ 60	else
Dimension / Overall	≥ 80	≥ 50	else

A basket of many metrics is judged more leniently than one metric alone — one weak check shouldn't turn the whole dimension red.

9. Measurement methods — not a single trick

Technique	Used by	Example
Structural graph traversal	CM-06, 07, 09, 12	asset → protection goal → threat → attacked interface
Matrix re-computation	CM-20	recompute risk from <code>RISK_MATRIX[impact]</code> <code>[likelihood]</code> and compare
Rule-based NLP (see next 2 slides)	CM-04, 17, 19, 24, 25–28	tokenization, lexicons, IDF weighting, Jaccard, slot-filling
Word-count thresholds	CM-14, 18, 31, 34	a real description has $\geq N$ words
Distribution analysis	CM-27	"rubber-stamping" — are all threats rated the same?

9a. Rule-based NLP — the full pipeline

The tool uses **classic (non-ML) NLP**. The text pipeline:

1. **Flatten** — `collect_text()` recursively gathers every string in the JSON.
2. **Lower-case / normalise** — `.lower()` , `_norm_rating()` (spaces → `_`).
3. **Tokenise** — `re.findall(r"[a-z0-9]{4,})"` → bag-of-words (4+ chars; 3+ for Jaccard).
4. **Stop-word removal** — subtract `GENERIC_THREAT_TERMS` (≈ 50 generic words like *attacker, system, access, data*) so only **concrete** terms remain.
5. **Match / score** — against lexicons, a good-threat vocabulary, or each other.

This is **lexicon + regex + set-theory NLP** — transparent and reproducible, with no training data or black-box model.

9b. Every NLP rule in the tool

#	NLP rule	Technique	Metric
1	Tokenisation → bag-of-words	<code>re.findall([a-z0-9]{4,})</code>	CM-17, 19, 24
2	Stop-word / generic-term removal	subtract <code>GENERIC_THREAT_TERMS</code>	CM-17
3	Good-threat vocabulary match	bag-of-words $\cap$ curated corpus + normalized IDF	CM-17
4	Heuristic specificity score	weighted 0.30/0.40/0.30 rule	CM-17

#	NLP rule	Technique	Metric
5	Jaccard set similarity	`	$A \cap B$
6	Placeholder lexicon (regex)	QUESTION, ??, TBD, TODO, test	CM-04
7	Vagueness / hedging lexicon	VAGUE_RE (<i>etc, might, some...</i>)	CM-17, CM-26
8	Actor / entity detection	ACTOR_RE (<i>attacker, insider...</i>)	CM-17 fallback signal
9	Approval / sign-off keywords	APPROVAL_RE (<i>approved, reviewed-by</i>)	CM-27

#	NLP rule	Technique	Metric
10	Evidence-artifact patterns	<code>EVIDENCE_RE</code> (<i>SRS, SCA, pen-test, CVE-####</i>)	CM-28
11	Word-count length gates	<code>len(re.findall(\w+)) ≥ N</code>	CM-13, 15, 17, 25

#	NLP rule	Technique	Metric
12	Semantic slot-filling	actor/action/target/weakness/impact/PG present	CM-25
13	Name ↔ keyword cross-match	normalised asset name $\subset$ PG name	Findings

9c. CM-17 additional fine-tuning (implemented)

CM-17 now uses an **IDF-weighted specificity check** instead of a raw term-overlap pass.

- Shared good-vocabulary terms are weighted by normalized IDF:

$$w(t) = \frac{\log(N/df_t)}{\log(N)}$$

where N is the number of good threats and df_t is the term's document frequency.

- A threat clears the concrete-language signal only if:
 - at least `CONCRETE_MIN_TERMS = 2` distinct terms match, and
 - the summed IDF strength is `>= CONCRETE_STRENGTH_MIN = 1.0`.

- Final CM-17 score combines three signals with weights:
 - interface linkage: 0.30
 - concrete language (IDF-weighted): 0.40
 - sufficient length + low vagueness: 0.30
- Pass threshold remains `SPECIFICITY_PASS_THRESHOLD = 0.65` .

This tuning makes keyword-stuffing with common words less effective while preserving transparent, rule-based behavior.

Evasion result (V13): With the updated IDF-weighted check (`CONCRETE_STRENGTH_MIN = 1.0`),

all 13 V13 stuffed threats correctly fail CM-17, making V13 a **true positive**.

With stronger V3/V9 mutations, detector performance over 13 variants is now:

TP=19 FP=2 FN=1 → precision=0.905 recall=0.950 F1=0.927.

In the mutation experiment, the primary artifact is a **delta-status matrix**: for each variant and criterion, we record the change in **status weight** relative to the clean reference TRA, i.e. $\Delta = w(V_i) - w(V_0)$ with **GOOD** = 100, **REVIEW** = 60, **CRITICAL** = 20.

A **true positive** means a targeted criterion's status weight drops under the injected defect;

an **off-target** drop is a false positive, and a targeted criterion that does not drop is a false negative.

10. Jaccard similarity — duplicate & inconsistent threats

Each threat's *name* + *attack description* becomes a **set of words**. Two threats are compared by overlap:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{\text{shared words}}{\text{total distinct words}}$$

- Result ranges **0** (nothing in common) → **1** (identical wording).
- Threshold: `JACCARD_DUP_THRESHOLD = 0.6` → **≥ 60 % shared = near-duplicate.**

```
jac = len(ta & tb) / len(ta | tb)
if jac >= 0.6:                # near-duplicate pair
    similar_pairs += 1
```

11. Jaccard — worked example

- Threat A words: {attacker, gains, root, access, ssh}
- Threat B words: {attacker, gains, admin, access, ssh}
- Shared ($\cap$) = {attacker, gains, access, ssh} = 4
- Union ($\cup$) = {attacker, gains, root, admin, access, ssh} = 6

$$J = \frac{4}{6} = 0.67 \geq 0.6 \Rightarrow \text{flagged as near-duplicate}$$

Two metrics use this:

- CM-24 — *counts* near-duplicate pairs (lower is better).
- CM-19 — checks if near-duplicates got the **same risk rating**; a different *high-severity* rating (Moderate/Major/Critical/Severe) = **inconsistency**.

12. CM-20 — rating coherence by matrix re-computation

Rather than trusting the stored risk value, the tool **independently recomputes** it:

$$\text{Likelihood} = f(\text{Exploitability}, \text{Exposure})$$

$$\text{Risk} = f(\text{Impact}, \text{Likelihood})$$

using the exact project matrices (`LIKELIHOOD_MATRIX` , `RISK_MATRIX`), then checks the stored value matches **cell-by-cell**.

-  Stored = computed **and** a rationale comment exists → pass.
-  Mismatch → the rating is internally **incoherent**.

13. Putting it together — the report

- **Hero score:** overall % + traffic light.
- **5 dimension cards:** sub-score per dimension.
- **Bar chart:** every CM metric's score.
- **Donut:** Good / Review / Critical counts.
- **Scorecard table:** each metric with note, interview trace, pass/fail readings.
- **Findings:** prioritised, actionable issues.

Refresh the browser to recompute after editing the JSON — fully reproducible.

13a. Out of scope — what the NLP does *not* do

The rule-based NLP is deliberately **transparent**, which means it has limits:

Not done	Consequence	
No semantic / ML understanding	synonyms & paraphrases not detected	
No stemming / lemmatisation	<i>exploit</i> ≠ <i>exploited</i> as tokens	
No language detection	assumes English text	
No spell-checking	typos lower specificity	

Not done	Consequence	
Word-count $\neq$ quality	a long but vague text can pass length gates	
Jaccard is lexical	reworded duplicates may slip under 0.6	

These are why many NLP metrics are **Semi-auto**: the tool *flags*, an expert *confirms*. The goal is a transparent, reproducible screen — not an AI oracle.

14. One-slide summary

1. Each metric → traffic light: GOOD / REVIEW / CRITICAL / N/A.
2. N/A = not applicable → *excluded* from all averages (never penalises).
3. Score = average of status weights: **GOOD = 100, REVIEW = 60, CRITICAL = 20.**
4. **Aggregate verdict:** ≥ 80 green, ≥ 50 amber, else red.
5. **Jaccard ≥ 0.6** = 60 %+ shared wording → near-duplicate; different serious rating → inconsistency.
6. **Methods:** graph traversal · matrix re-computation · rule-based NLP · regex · word-count · distribution analysis.

Every metric is **traceable to the expert interview question(s)** it was derived from.